Résolution du Problème du Voyageur de Commerce (TSP) par Algorithme Génétique

Métus Gbogbohoundada, Youssef Kassou, Steven Atitse

March 18, 2026

1 Contexte et Problématique

Ce projet de cours d'Intelligence Artificielle s'insère dans la mise en application des notions théoriques acquises au cours du semestre. Il s'agit d'une étude et d'une implémentation d'un algorithme génétique pour résoudre le problème du voyageur de commerce (TSP). Le problème du voyageur de commerce (TSP, *Traveling Salesman Problem*) consiste à déterminer, étant donné un ensemble de villes, le plus court chemin passant par chaque ville exactement une fois et revenant à la ville de départ. Il s'agit d'un problème classique d'optimisation combinatoire, utilisé comme référence dans de nombreuses études sur les algorithmes d'approximation et d'optimisation. Il est utilisé dans de nombreux domaines, notamment la logistique, la planification de tournées, et l'optimisation de réseaux.

2 Organisation du travail : Outils et Méthodes

2.1 Organisation

Pour le projet, le travail a été effectué en parallèle par chacun des membres du groupe et à la fin, une mise au propre a été faite pour harmoniser la production de tous. Cette approche s'est imposée, compte tenu des contraintes de disponibilité et donc de coordination, mais nous a permis de réaliser le travail dans les délais impartis.

2.2 Outils utilisés

Pour la réalisation de ce projet, nous avons utilisé les outils suivants :

- **Python** : pour l'implémentation de l'algorithme génétique et la manipulation des données.
- **Tkinter** : pour l'interface utilisateur.
- **Plotly** : pour la visualisation du circuit optimisé sur une carte.
- **Git et GitHub** : pour la gestion de versions et la collaboration.
- **JSON** : pour le stockage et la manipulation des données des villes.
- **LaTeX** : pour la rédaction du rapport.

3 Étude du problème

3.1 Complexité

Le problème de décision associé au TSP est **NP-Complet**. Il peut notamment être réduit à partir du problème du cycle Hamiltonien, un des 21 problèmes NP-Complets de Karp. Papadimitriou (1977) a démontré que le TSP reste NP-dur même si les distances sont euclidiennes. Ainsi, trouver la solution exacte devient rapidement intractable lorsque le nombre de villes augmente, rendant nécessaire l'utilisation d'algorithmes d'approximation tels que les algorithmes génétiques.

3.2 Analyse Combinatoire

Pour n villes, le nombre total de circuits possibles est $n!$. En fixant la ville de départ, le nombre de permutations reste $(n-1)!$. Tester exhaustivement toutes les permutations devient rapidement coûteux en ressources pour un nombre de villes supérieur à 10 (d'après nos tests), ce qui justifie l'utilisation d'approches heuristiques.

4 Implémentation de l'Algorithme Génétique

4.1 Modélisation

Pour l'implémentation de notre algorithme génétique, nous avons modélisé le TSP de la manière suivante :

- **Gène :** une ville, représentée par un nom, une latitude et une longitude.
- **Génotype :** ensemble des n gènes formant un individu.
- **Individu :** un tour du TSP, par exemple `[VilleA, VilleE, VilleC, VilleD]` si l'ensemble des villes à parcourir est `{VilleA, VilleC, VilleD, VilleE}`.
- **Population :** ensemble d'individus.

4.2 Paramètres de l'Algorithme

- **Fonction de Sélection :** *Tournoi.* On choisit aléatoirement k individus et l'on sélectionne les deux meilleurs selon la fonction *fitness*.
- **Fonction d'évaluation (fitness) :** La distance totale D du circuit, calculée en utilisant la formule de Haversine, qui prend en compte la courbure de la Terre pour calculer les distances entre les villes à partir de leurs coordonnées géographiques. Entre deux solutions, celle qui a la plus petite distance totale est considérée comme la meilleure. D s'exprime comme suit :

$$D = \sum_{i=1}^{n-1} d(v_i, v_{i+1}) + d(v_n, v_1)$$

  où $d(v_i, v_j)$ est la distance entre les villes v_i et v_j calculée par la formule de Haversine et v un individu.

4.3 Fonction de recombinaison

La recombinaison utilisée est le **Order Crossover (OX)** : une portion du parent est copiée telle quelle dans l'enfant, les villes manquantes sont ajoutées dans l'ordre du second parent en évitant les doublons.

4.4 Fonction de mutation

La mutation est une **swap mutation** avec une probabilité $p = 2\,\%$. Si un nombre q généré aléatoirement est inférieur à p, deux villes sont sélectionnées aléatoirement et échangées dans l'individu.

4.5 Stratégie de remplacement

La population suivante est constituée uniquement des enfants générés par celle qui l'a précédée : c'est un **remplacement générationnel**.

5 Paramètres des Tests

- Taille de la population (fixe pour toutes les générations) : 100 individus
- Nombre de générations : 500

6 Affichage et Visualisation

`NB` : notons que la gestion de l'affichage est réalisée dans un fichier tiers pour séparer la logique de notre algorithme de la partie interface utilisateur. Pour ce faire, les bibliothèques suivantes ont été utilisées :

- **Tkinter** pour l'interface utilisateur et l'affichage de la feuille de route.
- **Plotly** pour tracer le circuit optimisé sur une carte.

7 Résultats et Discussions

Notre algorithme principal fonctionne suivant l'algorithme représenté par le pseudo-code ci-dessous :

La recherche de la solution optimale par une approche de force brute est représentée par le pseudo-code ci-dessous :

Pour tester notre algorithme génétique, nous nous sommes servis de données réelles. Nous avons testé notre algorithme sur l'ensemble des communes de France (téléchargeables depuis l'adresse suivante : `https://geo.api.gouv.fr/communes?fields=nom,centre&format=json&geometry=centre`), en utilisant leurs coordonnées réelles. Nous avons évalué l'algorithme sur des ensembles de villes de tailles différentes (5, 10, 15, 20) pour évaluer sa performance et sa capacité à trouver des solutions proches de l'optimal.

La taille des populations à chaque génération était de 100 individus et l'algorithme a été exécuté sur 500 générations. Nous avons comparé les résultats obtenus par notre

Algorithm 1 Algorithme Génétique pour le TSP

```
procedure TSP_ALGORITHM(villes, taille_pop ← 100, gens ← 500)
    pop ← Generate_Population(villes, taille_pop)
    best ← arg min_{ind∈pop} Distance(ind)
    for i = 1 gens do
        pop ← New_Population(pop, taille_pop)
        current_best ← arg min_{ind∈pop} Distance(ind)
        if Distance(current_best) < Distance(best) then
            best ← current_best
        end if
    end for
    return best, Distance(best)
end procedure
```

Algorithm 2 Bruteforce pour la résolution du TSP

```
procedure BRUTE_FORCE(villes)
    best ← ∞
    best_tour ← null
    v_0 ← villes[0]                                  ▷ On fixe la ville de départ
    V_restantes ← villes \ {v_0}
    for all perm ∈ Permutations(V_restantes) do
        tour ← [v_0] + perm                          ▷ + désigne la concaténation de listes
        d ← Distance(tour)
        if d < best then
            best ← d
            best_tour ← tour
        end if
    end for
    return best, best_tour
end procedure
```

algorithme génétique avec les solutions exactes obtenues par une approche de force brute pour les instances de taille réduite ($n \leq 10$ villes).

Pour $n < 10$, l'algorithme trouve toujours la solution optimale ; pour $n = 10$, il trouve une solution à 1,5 % de l'optimal. Pour $n > 10$, l'approche combinatoire (force brute) est très lente pour le calcul de l'optimum. Nous pensons que les tests précédents attestent du bon fonctionnement de notre algorithme pour un nombre n de villes supérieur à 10.

8 Limitations et Perspectives

Pour améliorer notre algorithme et obtenir de meilleurs résultats, plusieurs pistes peuvent être envisagées :

- Introduire d'autres algorithmes de sélection (ex. *roulette wheel*) ou de mutation (ex. inversion) pour diversifier les solutions.
- Utiliser des techniques d'élitisme pour garantir la conservation des meilleures solutions d'une génération à l'autre.
- Tester sur des instances plus grandes ($n \geq 20$ villes) et comparer avec d'autres heuristiques (ex. algorithme des colonies de fourmis, recuit simulé).
- Utiliser la bibliothèque TSPLIB pour des benchmarks standardisés et comparer les performances de notre algorithme génétique avec d'autres approches publiées.
- Concevoir une bibliothèque plus généralisée pour le TSP, offrant une possibilité de paramétrage plus flexible pour d'autres besoins d'expérimentation.

9 Conclusion

Notre algorithme génétique permet de résoudre efficacement le TSP pour un grand nombre de villes en obtenant des solutions approchées. Le choix des paramètres (population, nombre de générations, méthodes de mutation et de croisement) influence directement la qualité des résultats. La comparaison avec la force brute confirme la pertinence de l'approche génétique pour l'optimisation combinatoire. De ce fait, de meilleurs choix de paramètres et d'autres techniques d'optimisation pourraient être explorés pour améliorer les performances et la qualité des solutions obtenues.